Multi-Agent Multiturn Scaffolding for Cost Effective Code Localization

Anonymous ACL submission

Abstract

Code localization refers to the task of identifying the files and functions that need to be edited to resolve a GitHub-style issue description. This is a critical step in automating software development and maintenance using coding agents. Strong localization is a prerequisite for strong code generation. However, localization requires inspecting large codebases, which consumes a large number of tokens, often exhausting the token budget. This is primarily because existing coding agents use terminal commands or tools designed to perform open-ended exploration of the codebase. Although this approach is feasible for large models with long contexts, it comes at the cost of huge token consumption and is almost impossible to fit in the context of smaller open source models.

To solve this, we propose a multi-agent system with a smarter codebase navigation scaffolding, that delegates token-heavy tasks of skimming through varoius code subgraphs to a smaller and cheaper post-trained agent (scorer) while working in unison with a larger and costlier agent to save its token consumption. We observe that such a collaborative system is not only cost effective but can augment any existing localization method and improve their performance across different benchmarks and programming languages.

1 Introduction

LLM coding agents increasingly resolve real GitHub issues by editing a repository directly, and benchmarks such as SWE-Bench now track how often they succeed (Jimenez et al., 2024). A patch can only be correct, however, if it changes the right code. Before any edit, the agent must identify the files and functions an issue actually touches, the step known as *code localization*. This step is decisive and largely unrecoverable: once a wrong location enters the agent's editable context, the repair stage reasons over the wrong code, and no later reasoning recovers the correct patch (Jimenez et al., 2024; Yang et al., 2024). Xia et al. (2024) drop agentic interaction and resolve issues with a fixed localize-then-repair pipeline, crediting much of their accuracy to a hierarchical localization phase, and methods that improve the localization stage report corresponding gains in end-to-end resolution (Sohrabizadeh et al., 2025). Localization is therefore not a preprocessing convenience but the step that determines how well downstream repair can perform.

Current systems close this gap in four ways. Reinforcement learning methods train the search policy itself: RepoSearcher pairs tool-integrated RL with rejection-sampled supervised fine-tuning (Ma et al., 2025), and SWE-RL scales RL over open software-evolution data (Wei et al., 2025). General-purpose agent scaffoldings give a capable LLM an open-ended action space with shell access, as in SWE-Agent and OpenHands (Yang et al., 2024; Wang et al., 2025). Graph-exploration agents such as LocAgent let an LLM traverse a repository code graph, hopping freely between related code units (Chen et al., 2025). Dense retrieval methods skip the agent altogether: SWERank scores code units with a bi-encoder retriever and a listwise LLM scorer (Reddy et al., 2025b).

The first three families pay for accuracy in tokens, and none of them bounds the cost. An RL agent, an open-ended scaffold, and a graph explorer all keep issuing tool calls and appending observations until they choose to stop, so a localization trajectory grows without a predictable ceiling. Under an OpenHands-style scaffolding, a single localization run can exceed 131K tokens (Section 3.4). A model with a long context window and ample compute absorbs that cost; the open 7–14B code models most teams can realistically run do not. A Qwen-class coder with a 32K-token window overflows before the search terminates. Reinforcement learning compounds the problem: a multi-step episode

returns only a sparse terminal reward, so adapting a small model to it takes many long and expensive rollouts. Dense retrieval is the one cheap option, but a single-step, order-centric ranker cannot revisit a discarded candidate or follow a structural edge, so it leaves the sequential, graph-structured side of the problem untouched. No existing method is at once bounded in token cost at inference and cheap enough to train on a small open model.

2 Method

2.1 Problem Setup

We represent a software repository as a code graph $\mathcal{G} = (V, E)$ whose nodes are code units (files, classes, functions) and whose edges encode structural relations such as *contains*, *invokes*, and *imports*. Given a natural-language issue q, a first-stage retriever induces a ranking π over V by relevance to q. Let $G \subseteq V$ denote the (unknown at test time) gold set of nodes that must change to resolve q. The task is to return a length-K list $\hat{L} \subseteq V$ that maximises

$$\text{recall@}K(\hat{L}) = \frac{|\hat{L} \cap G|}{|G|}. \quad (1)$$

The setting is bounded-context: a small open model cannot ingest the full ranked list, let alone the repository. The system observes only a window of r ranked nodes per round and may issue a bounded number of bounded-size model calls, precluding single-pass reranking of the candidate pool. We therefore cast localization as iterative set recovery.

2.2 Multi-Agent Multi-Turn Scaffolding

We split the work between a small post-trained *scorer* and a larger *agent*: the scorer triages a bounded candidate pool each round, and only the agent commits the length-K list. A *seed* is a node from which the scaffolding expands its graph neighborhood via BFS. One round runs in three steps (Figure 1); the loop runs for at most R rounds or until the ranked list is exhausted.

Step A: seed selection. At round t the scaffolding takes the next r unseen entries of the ranking π as the window W_t, and the scorer selects S seeds from W_t. If the scorer returns fewer than S identifiers we pad with the top-ranked remaining entries of W_t.

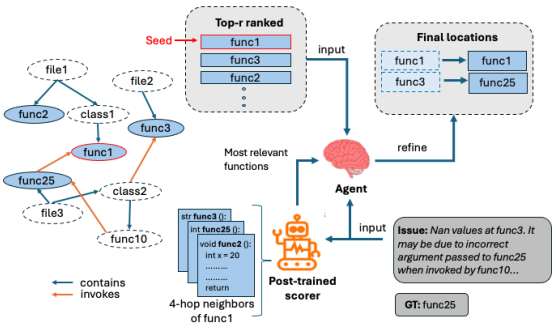

Figure 1: Overview of our multi-agent, graph-aware code localization method. A retriever returns a top-r ranked list of candidate functions; for each seed selected by the scorer, we traverse the code graph to collect its 4-hop neighbors, which the post-trained scorer ranks to select the most relevant subgraph nodes. An agent takes the issue, the ranked list, and these subgraph nodes as input and refines them into the final locations. In the example, the buggy function `func25` is recovered through graph expansion from the seed even though lexical ranking alone would miss it. One round is shown; the loop repeats for R rounds.

Step B: neighborhood expansion. For each of the S seeds the scaffolding runs a depth-d BFS along structural edges of the code graph (*contains* by default; ablations in Section 3.3 cover the four edge types). We keep only neighbors that also appear in the top-N of π (with N=500 throughout), a cheap rank pre-filter that removes graph noise without invoking the scorer. The scorer then keeps the n most relevant survivors per seed, yielding up to Sn neighbors $\mathcal{N}_t$.

Step C: list update. A single agent call sees the issue q, the current returned list $\hat{L}_{t-1}$ (initialised to the top-K of π at t=0), the window W_t, and the scorer-selected neighbors $\mathcal{N}_t$, and writes back the updated length-K list $\hat{L}_t$.

No call sees more than $r + Sn$ candidates plus $\hat{L}_{t-1}$, so per-call token cost is roughly constant across rounds and a 32K context comfortably absorbs it. Removing Steps A and B collapses the procedure to a pure window-update loop, giving a clean ablation that isolates the contribution of graph exploration (Section 3.3). The final prediction is $\hat{L} \equiv \hat{L}_R$.

2.3 Training the Scorer

The dense per-round shaping signal the scaffolding exposes telescopes to end-to-end recall@K, so by potential-based reward shaping (Ng et al., 1999) the dense per-round objective and the sparse end-

to-end metric share the same optimum; we state and prove the identity in Appendix C. Training the scorer therefore reduces to supervised learning on labelled candidate windows rather than reinforcement learning over multi-step trajectories. We parameterise the scorer as a low-rank adaptation (Hu et al., 2022) of Qwen2.5-Coder-3B-Instruct (Hui et al., 2024); only the adapter weights move during training, and no new scoring head is added. A standalone evaluation of the trained scorer against base Qwen and GPT-5.3 Codex baselines is reported in Appendix F.

Score readout. For an issue q and a candidate function c, we form a single chat-formatted prompt $x(q, c)$ that contains q, the token-truncated source of c, and an assistant turn asking whether c is relevant to q. Writing $\ell(x) \in \mathbb{R}^{|V|}$ for the next-token logits at the final position of x, we score c by the contrast between two fixed vocabulary tokens (ℓ_+ for the YES token, ℓ_- for NO),

$$\begin{aligned} s(q, c) &= \ell_+\big(x(q, c)\big) - \ell_-\big(x(q, c)\big), \\ p(q, c) &= \sigma\big(s(q, c)\big), \end{aligned} \tag{2}$$

with σ the logistic sigmoid. The language-model head stays frozen, so the readout adds no parameters.

Training groups. Each training instance is a single issue q together with a *group* of M candidates assembled to mirror the inference window. The group contains every ground-truth function for q that lies inside the retriever's top-N pool, completed with non-relevant candidates drawn to span the full rank depth: a fixed fraction is taken as hard negatives from the top of the pool, and the remaining slots are filled by one-per-bin sampling across the deeper pool. Members are position-shuffled so order does not leak the label, and negatives are resampled at the start of every epoch (Appendix E).

Objective. Let $s_j := s(q, c_j)$ and $y_j \in \{0, 1\}$ be the label of c_j, and write $\mathcal{G} = \{j : y_j = 1\}$ for the indices of the ground-truth members. The loss combines a pointwise binary cross-entropy under the sigmoid link of (2),

$$\begin{aligned} \mathcal{L}_{\text{BCE}} = -\frac{1}{M} \sum_{j=1}^{M} \big[& y_j \log \sigma(s_j) \\ & + (1 - y_j) \log(1 - \sigma(s_j)) \big], \end{aligned} \tag{3}$$

with a listwise *coverage* term — a softmax log-likelihood over the group, evaluated at each ground-truth member and averaged,

$$\mathcal{L}_{\text{cov}} = -\frac{1}{|\mathcal{G}|} \sum_{j \in \mathcal{G}} \log \frac{\exp(s_j)}{\sum_{k=1}^{M} \exp(s_k)}. \tag{4}$$

The full objective is the convex combination $\mathcal{L} = \mathcal{L}_{\text{BCE}} + \lambda \mathcal{L}_{\text{cov}}$. $\mathcal{L}_{\text{BCE}}$ supplies a gradient on every candidate and calibrates the per-candidate probabilities that downstream selection and the next epoch's hard-negative mining read off the score. $\mathcal{L}_{\text{cov}}$ is permutation-invariant across $\mathcal{G}$ and concentrates softmax mass on ground-truth members relative to in-group negatives, without penalising gold-vs-gold order. Setting λ=0 recovers a pure-BCE scorer, which we report as an ablation alongside the combined loss (Section 3.3). Top-K selection itself is performed only at inference and is never differentiated.

3 Experiments

3.1 Setup

Datasets. The scorer is trained on a fixed subset of 5,000 issues drawn from the SWE-Bench train split (Jimenez et al., 2024) and evaluated on SWE-Bench Verified (Jimenez et al., 2024) — the human-verified subset restricted to issues with unambiguous patches — and SWE-PolyBench (Chaudhari et al., 2025), a multilingual SWE-Bench-style benchmark spanning Python, Java, JavaScript and TypeScript. Across all three splits we recover the ground-truth function set G_i from the PR commit that resolved issue i: every function whose body the patch modifies is treated as gold. Group construction and negative sampling at training time follow Appendix E.

Backbones and baselines. The scorer is fine-tuned from Qwen2.5-Coder-3B-Instruct (Hui et al., 2024). As un-fine-tuned reference points under the same scaffolding we report Qwen2.5-Coder-14B-Instruct, Qwen3-Coder-30B-A3B-Instruct (Yang et al., 2025), and GPT-5.3 Codex. The first-stage retriever is either SWERankEmbed-Large (Reddy et al., 2025b) (dense) or BM25 (lexical), in both cases truncated to a top-N pool with N=500. The zero-shot retriever output and the un-fine-tuned scorer under the same scaffolding act as ablation baselines.

3.2 Main Results

Table 1 reports Recall@5 on SWE-Bench Verified (N=500) for the zero-shot retrievers, the un-fine-tuned baselines, and our fine-tuned scorer, under both first-stage retrievers. The block labelled *w/o graph* is the ablation in which Step B is disabled (Section 2.2), so the loop reduces to window-only refinement; the *full* block runs the complete three-step round.

Three observations stand out. *First*, the zero-shot retrievers plateau near 0.35 Recall@5; the LLM-driven refinement loop closes a substantial part of that gap even without any training. *Second*, adding graph exploration (Step B) lifts every model in the table, with the largest absolute jumps on small backbones: Qwen2.5-Coder-14B-Instruct rises from 0.354 to 0.435 under BM25 retrieval, and Qwen3-Coder-30B-A3B-Instruct moves from 0.502 to 0.549. Total token consumption rises roughly 3× with exploration, driven by the higher call count even as per-call context shrinks. *Third*, the smallest open backbones still trail GPT-5.3 Codex (which reaches 0.559 under the full scaffolding), motivating the fine-tuned scorer whose results occupy the last row.

Standalone scorer analysis. To isolate the scorer from the multi-round scaffolding, Figure 2 reports function-level recall on SWE-PolyBench under a single-pass windowed protocol: the first-stage retriever returns the top PS=100 candidates, the scorer ranks a window of K=20, and the top-L=5 by score are selected. The fine-tuned 3B scorer dominates both 0.5B variants and closes most of the gap to GPT-5.3 Codex across languages despite being two orders of magnitude smaller. File-level recall and a calibration diagnostic (p_{gt} vs. $p_{\neg gt}$) under the same protocol are deferred to Appendix F.

End-to-end results on SWE-PolyBench. Table 2 reports end-to-end Recall@5 and Accuracy@5 (perfect recall) on the SWE-PolyBench Java and Python splits, comparing the scaffolding without graph exploration (Step B disabled) against the full scaffolding with our fine-tuned scorer. The full scaffolding improves both metrics on both languages: on Java, GPT-5.3 Codex gains +5 recall and +5 accuracy points; on Python, Qwen3-Coder-30B gains +5 recall and +4 accuracy points and surpasses the GPT-5.3 Codex baseline. Token consumption stays comparable to the no-graph variant, so the gains come essentially free of extra compute.

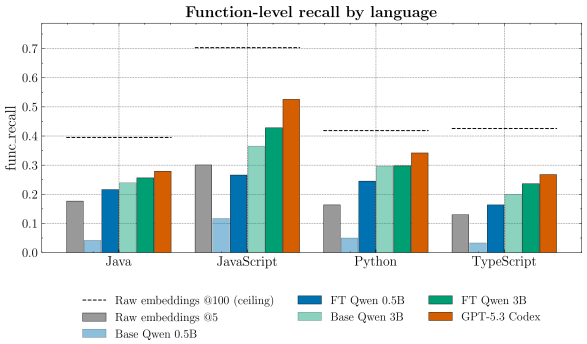

Figure 2: Per-language function-level recall on SWE-PolyBench under a windowed scorer protocol (pool size PS=100, candidate window K=20, top-L=5). The dashed line is the retrieval ceiling at PS=100. Comparisons include the raw embedding ranker at L=5, base and fine-tuned Qwen2.5-Coder 0.5B/3B-Instruct, and GPT-5.3 Codex. The fine-tuned 3B scorer dominates the 0.5B variants and trails GPT-5.3 Codex by 2–10 points across languages; JavaScript shows the largest residual gap, where GPT-5.3 Codex benefits most from its longer context.

3.3 Does the Coverage Term Help?

3.4 Efficiency and Scaling

4 Discussion and Conclusion

Discussion. The picture across SWE-Bench Verified (Table 1) and SWE-PolyBench (Table 2, Figure 2) is consistent. First, a bounded-context scaffolding closes most of the gap left by the first-stage retriever: every LLM-driven configuration we tried exceeds the ≈ 0.35 Recall@5 ceiling of the zero-shot ranker. Second, graph exploration (Step B) is the largest single lever, lifting recall and accuracy on every model and language while keeping per-call context short. Third, a small fine-tuned scorer is enough to drive the full scaffolding: a Qwen2.5-Coder-3B scorer matches or beats Qwen3-Coder-30B-A3B under the same scaffolding on Verified, and pairs with a 30B Step-C agent to set the best Python numbers on SWE-PolyBench. Token consumption tracks the call count rather than the model size, so larger Step-C agents do not pay a context tax for using our scorer.

Key takeaways. (i) Localization scales better through scaffolding structure than through model size: a 3B scorer plus a 30B agent under our loop beats a 30B agent alone. (ii) Graph-aware exploration is cheap because the scorer sees short neighborhood windows rather than the full pool; the cost of an extra step is small compared to the recall it recovers. (iii) Supervised training is sufficient for

Table 1: Retrieval performance on SWE-Bench Verified (N=500). Recall@5 is reported separately under a dense (SWERankEmbed-Large) and a lexical (BM25) first-stage retriever. Token consumption is the total tokens per issue (tokens/call × #calls), averaged across the two retrievers. *Zero-shot rows evaluate the raw retriever ranking at K=20, since no LLM-based pruning is applied.

		Recall@5		
Method	Model	Dense	BM25	Tokens (k)
Zero-shot	SWERankEmbed-Large	0.35*	–	–
	BM25	–	0.36*	–
w/o graph	GPT-5.3 Codex	0.51	–	63
	Qwen2.5-14B-Instruct	0.26	0.29	73
	Qwen2.5-Coder-14B-Instruct	0.35	0.35	69
	Qwen3-Coder-30B-A3B-Instruct	0.50	0.50	70
	+ Qwen2.5-Coder-3B scorer (ours)	**0.45**	–	**33**
w/ graph, w/o scorer	GPT-5.3 Codex	0.56	0.56	204
	Qwen2.5-14B-Instruct	0.36	0.36	204
	Qwen2.5-Coder-14B-Instruct	0.44	0.42	198
	Qwen3-Coder-30B-A3B-Instruct	0.54	0.55	271
w/graph, w/ scorer	**Ours, fine-tuned Qwen2.5-Coder-3B**	**0.55**	–	**35**

Table 2: Retrieval performance on SWE-PolyBench Java and Python under two Step-C agents (GPT-5.3 Codex and Qwen3-Coder-30B-A3B-Instruct). Recall@5 and Accuracy@5 (perfect recall) are reported after filtering out instances with empty function-level GT and using denominator $|GT \cap \text{graph\_functions}|$. "–" marks a run we have not executed. Tokens/call is averaged across the kept instances; #Calls is per issue.

Java (kept = 158 / 165)

Model	Rec@5	Acc@5	Tok/call	#Calls
w/o graph (Step B disabled)				
GPT-5.3 Codex	0.38	0.22	5597	3.87
Qwen3-Coder-30B-A3B	0.35	0.20	5847	4.75
Full, ours (Qwen2.5-Coder-3B scorer)				
GPT-5.3 Codex	**0.43**	**0.27**	5504	3.47
Qwen3-Coder-30B-A3B	0.37	0.20	5871	4.56

Python (kept = 182 / 199)

Model	Rec@5	Acc@5	Tok/call	#Calls
w/o graph (Step B disabled)				
GPT-5.3 Codex	–	–	–	–
Qwen3-Coder-30B-A3B	0.40	0.29	8099	4.70
Full, ours (Qwen2.5-Coder-3B scorer)				
GPT-5.3 Codex	0.42	0.31	9242	3.34
Qwen3-Coder-30B-A3B	**0.45**	**0.33**	9778	4.75

the scorer because the per-round shaping signal telescopes to end-to-end Recall@K, so no policy-gradient machinery is needed for the inner loop.

Conclusion. We cast code localization as sequential set-coverage on a code graph and built a bounded-cost, multi-round, multi-agent scaffolding around that view. The round structure exposes a dense per-round target whose optimum coincides with end-to-end Recall@K, which lets a small open scorer be trained by ordinary supervised learning rather than sparse-reward RL. The resulting Qwen2.5-Coder-3B scorer makes the full scaffolding competitive with GPT-5.3 Codex on SWE-Bench Verified and on SWE-PolyBench Java and Python, at a fraction of the token cost. Stop-timing and cross-round coordination remain natural candidates for a light-weight RL stage on top of the supervised scorer; we leave that to future work.

Limitations

Partial multilingual coverage. Our end-to-end scaffolding results on SWE-PolyBench (Table 2) cover only the Java and Python splits at submission time; the JavaScript and TypeScript end-to-end runs are not yet complete. The standalone scorer evaluation (Figure 2) does span all four languages, so the trained scorer itself has been measured broadly, but the full Step-A/B/C pipeline numbers for JavaScript and TypeScript are pending.

Benchmark scope. All evaluations are on SWE-Bench-family benchmarks (SWE-Bench Verified and SWE-PolyBench). These cover human-curated GitHub issues drawn mostly from popular open-source libraries; generalization to private-codebase distributions, proprietary languages, or non-bug-fix maintenance tasks (refactors, feature additions) is untested.

Graph construction is assumed away. The scaffolding requires a pre-built code graph $\mathcal{G}$. We reuse the graphs released with prior work and construct new ones with the same pipeline (Appendix D), but the construction cost, the failure modes for languages with weaker Tree-sitter coverage, and the sensitivity of recall to graph quality are not studied here. A noisy or incomplete graph is a silent failure mode that this paper does not characterise.

Surrogate coverage term. The listwise term $\mathcal{L}_{\text{cov}}$ in (4) is a differentiable softmax surrogate, not the exact (non-differentiable) set-coverage objective. Our argument guarantees that the dense per-round target shares the same optimum as end-to-end Recall@K; whether $\mathcal{L}_{\text{cov}}$ attains that optimum exactly, rather than only approximating it, is not proven.

Single scorer backbone. We instantiate the scorer only on Qwen2.5-Coder-3B-Instruct (with a 0.5B variant reported as an ablation). How the recipe scales to other base families (Llama-Coder, DeepSeek-Coder, non-Coder instruction models) or to substantially larger adapters is not evaluated.

Stop-timing and cross-round coordination. The scaffolding always runs to the round budget or pool exhaustion; an early-stop hook is left for an RL stage that we do not train or evaluate in this paper. Likewise, cross-round coordination between the scorer and the agent uses a fixed protocol and is not learned. Both are natural targets for a follow-up that adds a thin RL layer on top of the supervised scorer.

References

Shravan Chaudhari, Rahul Thomas Jacob, Mononito Goswami, Jiajun Cao, Shihab Rashid, and Christian Bock. 2025. SpIDER: Spatially informed dense embedding retrieval for software issue localization. *arXiv preprint arXiv:2512.16956*.

Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. 2025. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8697–8727, Vienna, Austria. Association for Computational Linguistics.

Fabio James Fehr, Prabhu Teja S, Luca Franceschi, and Giovanni Zappella. 2025. CoRet: Improved retriever for code editing. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 775–789, Vienna, Austria. Association for Computational Linguistics.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, Online. Association for Computational Linguistics.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. Graphcodebert: Pre-training code representations with data flow. *Preprint*, arXiv:2009.08366.

Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, and 5 others. 2024. Qwen2.5-coder technical report. *Preprint*, arXiv:2409.12186.

Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. 2025. Cosil: Software issue localization via llm-driven code repository graph searching. *Preprint*, arXiv:2503.22424.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Zexiong Ma, Chao Peng, Qunhong Zeng, Pengfei Gao, Yanzhen Zou, and Bing Xie. 2025. Tool-integrated reinforcement learning for repo deep search. *arXiv preprint arXiv:2508.03012*.

Andrew Y Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning*, pages 278–287.

Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2025. Repograph: Enhancing AI software engineering with repository-level code graph. In *The Thirteenth International Conference on Learning Representations*.

Revanth Gangi Reddy, Ye Liu, Wenting Zhao, Jae-Hyeok Doo, Tarun Suresh, Daniel Lee, Caiming Xiong, Yingbo Zhou, Semih Yavuz, and Shafiq Joty. 2025a. SweRank+: Multilingual, multi-turn code ranking for software issue localization. *arXiv preprint arXiv:2512.20482*.

Revanth Gangi Reddy, Tarun Suresh, JaeHyeok Doo, Ye Liu, Xuan Phi Nguyen, Yingbo Zhou, Semih Yavuz, Caiming Xiong, Heng Ji, and Shafiq Joty. 2025b. Swerank: Software issue localization with code ranking. *Preprint*, arXiv:2505.07849.

Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. 1994. Okapi at trec-3. In *Text Retrieval Conference*.

Atefeh Sohrabizadeh, Jialin Song, Mingjie Liu, Rajarshi Roy, Chankyu Lee, Jonathan Raiman, and Bryan Catanzaro. 2025. Nemotron-CORTEXA: Enhancing LLM agents for software engineering tasks via improved localization and solution diversity. In *Forty-second International Conference on Machine Learning*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2025. Openhands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*.

Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. 2025. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*.

Dejiao Zhang, Wasi Uddin Ahmad, Ming Tan, Hantian Ding, Ramesh Nallapati, Dan Roth, Xiaofei Ma, and Bing Xiang. 2024. CODE REPRESENTATION LEARNING AT SCALE. In *The Twelfth International Conference on Learning Representations*.

Zhaoxi Zhang, Yitong Duan, Yanzhi Zhang, Yiming Xu, Zhixiang Wang, Kun Liang, Yang Li, Jiahui Liang, Deguo Xia, Jizhou Huang, Jiyan He, and Yunfang Wu. 2025. One tool is enough: Reinforcement learning for repository-level LLM agents. *arXiv preprint arXiv:2512.20957*.

Albert Örwall. 2024. Moatless tools.

A Notation

Table 3 collects the symbols used in the paper, grouped by the section of first use. The symbol $\mathcal{G}$ denotes the code graph in Section 2.1 and an index set of ground-truth members during scorer training in Section 2.3; the two uses do not co-occur and context disambiguates.

B Related Work

Reinforcement learning for localization. RepoSearcher equips a repository agent with a curated tool set and trains it through rejection-sampled supervised fine-tuning followed by tool-integrated RL, reaching strong localization with a 32B model (Ma et al., 2025). SWE-RL scales RL over open software-evolution data and shows the learned reasoning transfers across software tasks (Wei et al., 2025). RepoNavigator narrows the agent to a single jump-to-definition tool and trains end-to-end with RL from a base model, so that a 7B agent can outperform substantially larger baselines (Zhang et al., 2025). In every case training optimizes multi-step episodes that return only a sparse terminal reward, so adaptation is costly in rollouts and compute. Our round decomposition exposes a dense per-round target that telescopes to recall@K (Section 2.3; proof in Appendix C), turning the same problem into ordinary supervised fine-tuning.

Agentic scaffolding. SWE-Agent and OpenHands wrap a strong LLM in a general-purpose environment with shell access and an open-ended action space, and they set much of the standard for end-to-end issue resolution (Yang et al., 2024; Wang et al., 2025); Moatless provides a comparable tool-driven loop (Örwall, 2024). Agentless instead drops interaction and runs a fixed localize-then-repair pipeline, showing that a simple prompted procedure can rival full agents (Xia et al., 2024). The interactive scaffoldings place no ceiling on trajectory length, so token cost is unbounded and a localization run easily overflows the context window of the open 7–14B models we target. Agentless is bounded, but its localization is a fixed prompted

Symbol	Meaning
Problem setup (Sec. 2.1)	
$\mathcal{G}=(V, E)$	Code graph: V are code units, E are structural edges (*contains*, *invokes*, *imports*, *inherits*)
q	Natural-language issue description
π	Retriever-induced ranking over V
$G \subseteq V$	Gold set: nodes that must change to resolve q
K	Fixed length of the returned list
$\hat{L}$	Predicted length-K list, $\hat{L} \subseteq V$
Scaffolding (Sec. 2.2)	
t, R	Round index; round budget (max rounds)
r	Window size: candidates revealed per round
W_t	Round-t window: next r unseen entries of π
d	BFS depth (Step B)
N	Retriever pool size for the Step B pre-filter (N=500)
S	Seeds (BFS centers) per round
n	Max neighbors per seed kept by the scorer
$\mathcal{N}_t$	Scorer-kept neighbors at round t; $\lvert\mathcal{N}_t\rvert \leq Sn$
$\hat{L}_t$	Running predicted list at round t; $\hat{L} \equiv \hat{L}_R$
Scorer training (Sec. 2.3)	
c, c_j	Candidate function
$x(q, c)$	Chat prompt fed to the scorer
ℓ_+, ℓ_-	Logits at the YES / NO vocabulary tokens
$s(q, c)$, s_j	Scorer score, $\ell_+ - \ell_-$
$p(q, c)$	$\sigma(s(q, c))$, sigmoid probability
y_j	Relevance label of c_j
M	Group size: candidates per training instance
$\mathcal{G}$	Index set of GT members in a training group, $\{j : y_j{=}1\}$ (overloads the code-graph symbol)
$\mathcal{L}_{\text{BCE}}$, $\mathcal{L}_{\text{cov}}$	Pointwise BCE; listwise coverage loss
λ	Coverage weight (λ=0.2 main; λ=0 ablation)
w_+, w_-	BCE class weights (positive; negative)

Table 3: Notation used throughout the paper. Each symbol is also defined at its point of first use in the body.

pipeline, not a component a small model can learn to improve. Our scaffolding bounds the token cost of every round and stays trainable, so a small model can run the search within budget and be trained to localize better.

Graph-based exploration. A related line hands the LLM the repository's structure. LocAgent builds a heterogeneous code graph and lets the model explore it through graph-traversal actions (Chen et al., 2025), CoSIL has the model search a call graph it expands on the fly without a pre-built index (Jiang et al., 2025), and RepoGraph supplies a repository-level code graph for software-engineering agents (Ouyang et al., 2025). This work establishes that structural edges carry localization signal. The traversal, however, is unconstrained, so trajectory length and token cost again grow without a bound. We use the same structure through bounded neighborhood expansion around ranked seeds, so graph signal raises recall without inflating the budget.

Retrieval and reranking. Localization is also posed as retrieval. Lexical matching (Robertson et al., 1994) and learned code embeddings (Feng et al., 2020; Guo et al., 2021; Zhang et al., 2024) score code units against the issue text, and SWERank sharpens this for issue localization with a bi-encoder retriever and a listwise LLM scorer (Reddy et al., 2025b); CoRet improves the retriever for code-editing queries (Fehr et al., 2025) and SpIDER incorporates repository structure into the dense representation (Chaudhari et al., 2025). These rankers are cheap and bounded, but the objective is per-query and order-centric, optimizing the order of one list rather than coverage of the gold set, and a single pass cannot revisit a discarded unit or follow a structural edge. SWERank+ adds a multi-turn agentic loop with a memory buffer (Reddy et al., 2025a), yet the loop is open-ended and the training signal stays a ranking objective. We recast localization as sequential set-coverage on a code graph and train an objective provably co-optimal with recall@K (Section 2.3), keeping the bounded cost of retrieval while adding the set recovery and multi-round refinement that ranking omits.

C Proof of the Telescoping Identity

D Code Graph Construction

The scaffolding's neighborhood expansion (Section 2.2, Step B) walks a per-repository code graph $\mathcal{G}=(V, E)$. We follow the four-edge schema of Chen et al. (2025) as implemented in Chaudhari et al. (2025), summarised here for completeness.

For each repository the graph is built from the source files written in the repository's primary language; files in secondary languages are ignored. Nodes represent code entities at four granularities (directory, file, class, and function), and edges encode four structural relations: a *contains* edge linking each container to the code units it declares, an *invokes* edge between a function and each function it calls, an *imports* edge between a file and each

module it imports, and an *inherits* edge between a class and each parent class it extends. Every non-directory node stores its source text alongside its identifier, so the model can see code when a node is shown.

We extract Python source files with the standard `ast` module and Java, JavaScript, and TypeScript files with Tree-sitter[1]. The traversal of the repository's directory tree adds directory nodes for navigation, skips files that contain no class or function definition, and emits one node per declared entity together with its containment, call, import, and inheritance edges. Multi-file class definitions and the more flexible declaration patterns of the non-Python languages are handled with language-specific Tree-sitter queries; we refer to Chaudhari et al. (2025) for the full list of cases. Graphs are constructed on demand at the start of a localization run and reused across the rounds.

For the experiments in Section 3, we use the graphs released with Chaudhari et al. (2025) for SWE-PolyBench and SWE-Bench Verified, and construct graphs from scratch for the SWE-Bench Lite dev split with the same pipeline.

E Scorer Training Details

This appendix complements Section 2.3 with the hyperparameters and procedural details that we omit from the main body.

Group construction. For each issue, every ground-truth function that falls inside the retriever's top-N pool is included as a positive; the group is then completed to size M with non-relevant candidates. Negatives are drawn with a stratified strategy: a fixed fraction is taken as top-ranked hard negatives, and the remaining slots are filled by partitioning the rest of the pool into equal-width rank bins and sampling one negative per bin. Each group therefore exposes the model to shallow, mid- and deep-pool negatives, which empirically prevents the calibration drift on deep-pool non-relevant candidates we observed under top-only hard-negative sampling. Negatives are re-sampled at the start of every epoch, so depth coverage widens monotonically across training.

Optimization. The adapter is trained in bfloat16 with FlashAttention-2 and activation checkpointing on the backbone. AdamW is used with the schedule of Table 4; gradients are accumulated over 8 groups before each optimizer step and clipped to unit norm. All M candidates of a group are forwarded under autograd in a single optimizer iteration so that the listwise term in (4) backpropagates jointly across the group.

Distributed training. Training is data-parallel across GPUs via Hugging Face Accelerate. At the start of each epoch the shuffled issue list is sharded into disjoint contiguous slices — one per rank — and each rank builds its own training groups independently. Because issues whose ground-truth functions fall outside the retriever pool are dropped, the per-rank post-filter group counts are reduced with a min-collective to keep DDP collectives aligned. LoRA gradients are small relative to the backbone, so all-reducing them on every backward is inexpensive and we do not shard the model.

Evaluation during training. After each epoch the current adapter is evaluated on a fixed held-out subset of validation issues using the same windowed top-K procedure as inference: candidates are scored in sliding windows over the top-N pool, the global top-K by score is selected, and function- and file-level recall@K are reported. Best-epoch selection on this validation metric determines the adapter shipped for the test-time evaluation in Section 3.

Prompt format. The training-time prompt format matches the inference scorer's format exactly, including the assistant-turn template and the position at which the YES/NO logits are read. Mismatches between train- and test-time prompts silently shift the score distribution and are the most common source of regression when porting the adapter to a new backbone.

F Additional Results

The figures below complement the function-level scorer evaluation reported in Section 3.2 (Figure 2). Both use the same windowed protocol on SWE-PolyBench: the first-stage retriever returns the top PS=100 candidates, the scorer ranks a window of K=20, and the top-L=5 by score are selected.

G AI Disclosure

[1] https://github.com/tree-sitter

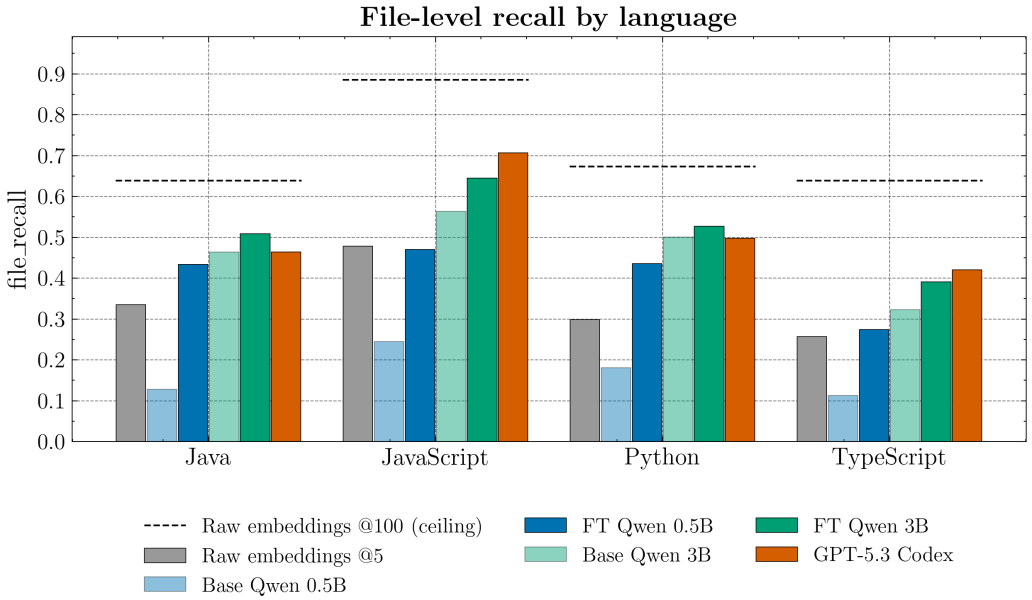

Figure 3: Per-language file-level recall on SWE-PolyBench under the windowed protocol of Figure 2. The dashed line marks the retrieval ceiling — the recall achievable if the gold function set lies in the top-100 retriever pool. The fine-tuned 3B scorer (FT Qwen 3B) matches or exceeds GPT-5.3 Codex on Java and Python and tracks it within a few points on JavaScript and TypeScript.

Table 4: Scorer fine-tuning hyperparameters. The same configuration is used for the BCE-only (λ=0) ablation and the main BCE+coverage (λ=0.2) runs.

Component	**Value**
Backbone	Qwen2.5-Coder-3B-Instruct (Hui et al., 2024)
Precision	bfloat16, FlashAttention-2
Adapter	LoRA, r=4, α=32, dropout 0.05
LoRA targets	$\{q, k, v, o\}_{\text{proj}}$, $\{\text{gate}, \text{up}, \text{down}\}_{\text{proj}}$
Score readout	$s = \ell(\text{yes}) - \ell(\text{no})$ at final prompt position
Group size M	20 candidates per issue
Issue / code trunc.	3072 / 1024 tokens
Negative sampling	40% top hard negatives; remainder one-per-bin across pool; re-sampled each epoch
BCE class weights	w_+=w_-=1
Coverage weight	λ=0.2 (main); λ=0 (ablation)
Optimizer	AdamW
Learning rate	1×10^{-4}, cosine, 3% warm-up
Grad. accumulation	8 groups per step
Grad. clipping	max norm 1.0
Epochs	5
Retriever pool N	500
Train / val split	95%/5% of issues, fixed seed

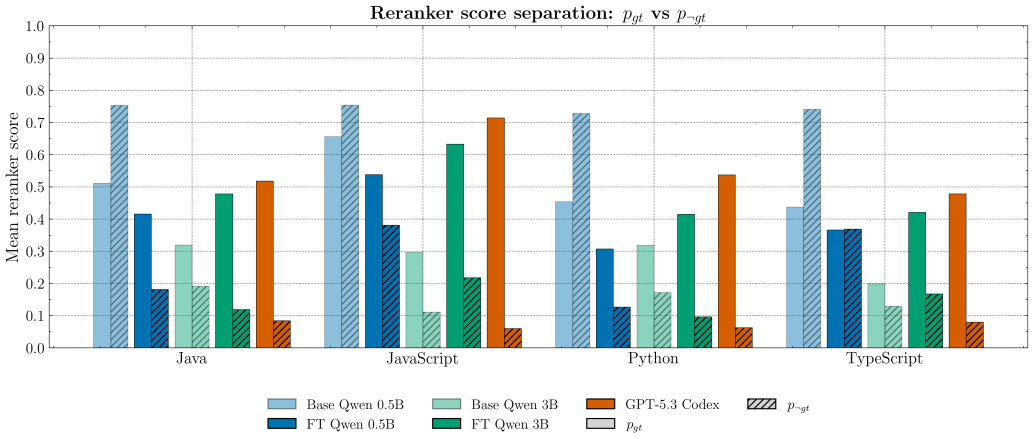

Figure 4: Mean score on ground-truth candidates (p_{gt}, filled bars) vs. non-relevant candidates ($p_{\neg gt}$, hatched bars), averaged per language under the same windowed protocol. A wider gap indicates better discrimination between gold and non-gold. Fine-tuning widens the separation at every model size: the base 0.5B model is actively mis-calibrated — it assigns $p_{\neg gt} > p_{gt}$ in every language — whereas the fine-tuned 3B variant achieves a separation comparable to GPT-5.3 Codex, with p_{gt} above $p_{\neg gt}$ by a margin of 0.25–0.45.